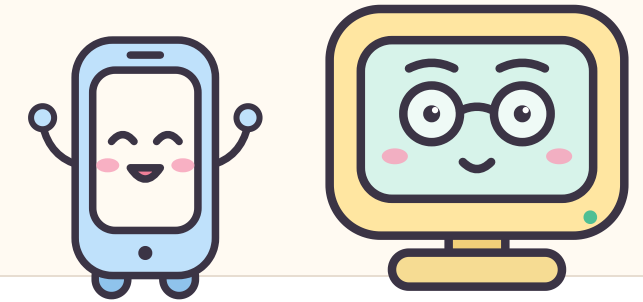




第 3 集 · 约 26 分钟

协议①：加密信道

密文里改掉一个字节，也能被发现

配套阅读：书的第 2 章



从零写出 LanShare

这一集你会学到

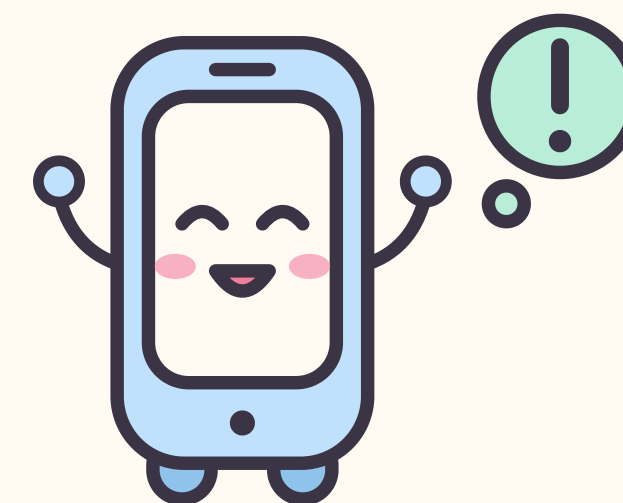
1. AEAD：加密和防篡改一步完成，附加数据把密文绑在接口上
2. nonce 绝不能重复：用请求计数器拼出 nonce
3. 借用、Vec、Result、enum、trait，第一次全部见到
4. 集成测试：篡改任何一个字节，都必须解不开



先想一个问题

用 v1 传一张照片，同一个 WiFi 下的人看得到吗？

看得到。v1 走的是明文 HTTP，抓包就能看到文件内容和文字，连扫码网址里的口令都是明文。



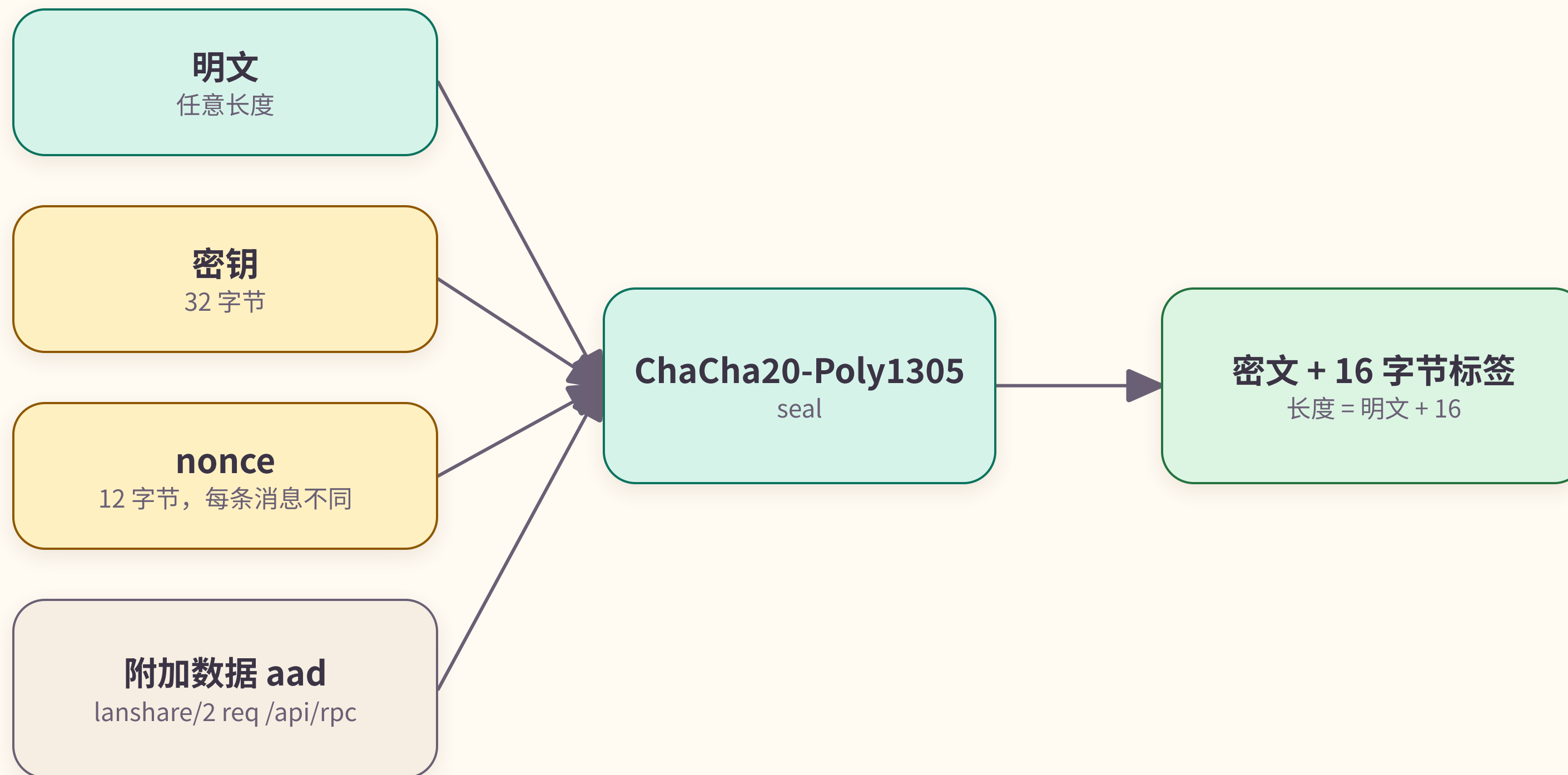


加密信道要做到四件事

- 看不懂：抓到的全是乱码
- 改不了：改动任何一个字节，接收方都能发现
- 挪不走：发给上传接口的密文，挪到别的接口就作废
- 不能重放：同一段密文原样再发一遍，也要拒绝

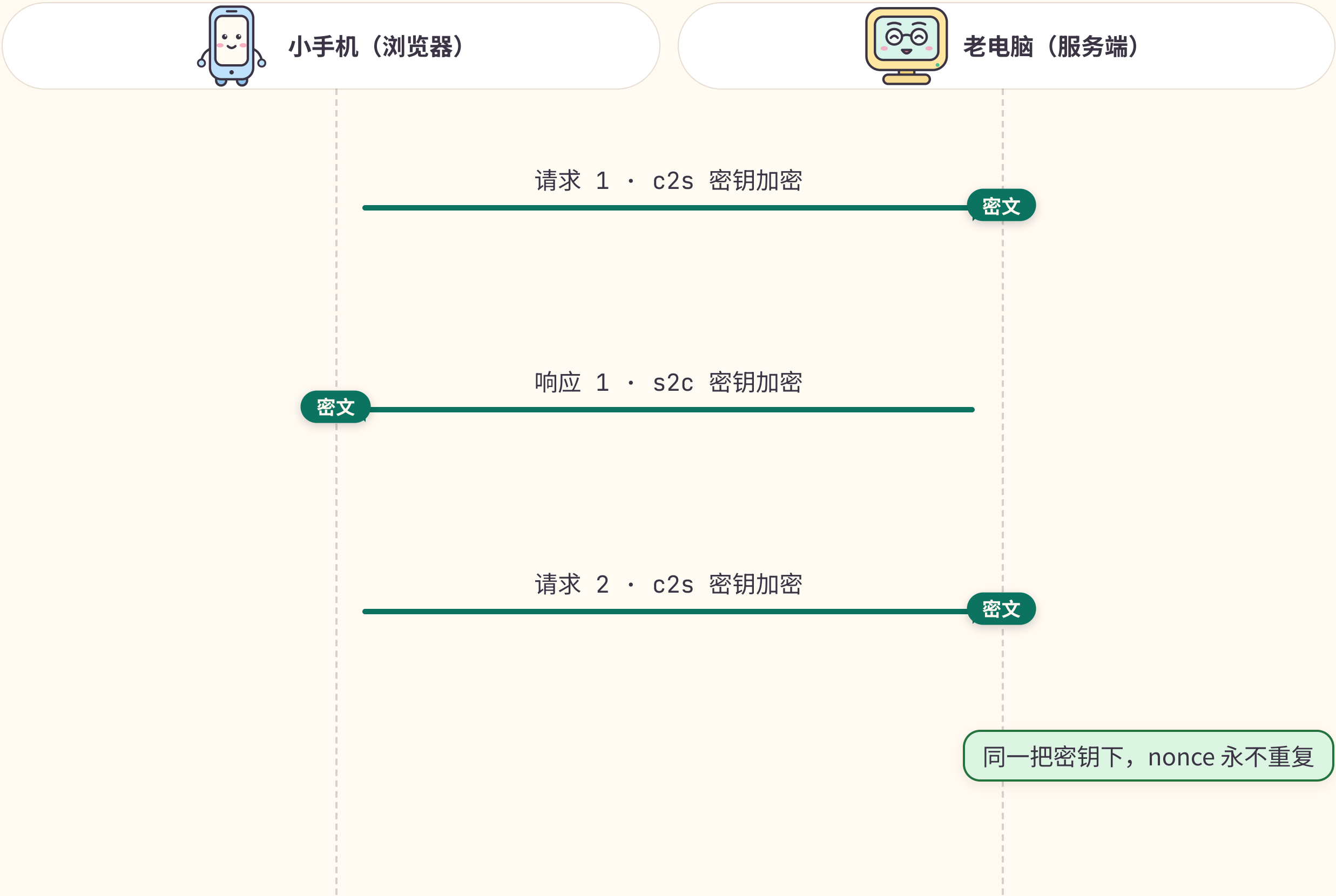


AEAD：一次调用，加密加防篡改





nonce 从哪来：请求计数器



发生了什么

nonce = 4 个零字节 + 计数器 1

nonce 和请求一样，但换了一把密钥

计数器 +1，nonce 跟着变



channel.rs: 文件说明和引入

channel.rs

crates/lanshare-proto/src/

第 1-7 行 · 共 7 行

```
1  //! 加密信道: ChaCha20-Poly1305, nonce 由请求计数器构成。
2  //!
3  //! 每个方向一把密钥 (c2s / s2c), 每个请求的计数器唯一, 所以 (密钥, nonce) 永不重复—
4  //! 这是 ChaCha20-Poly1305 安全性的前提。
5
6  use chacha20poly1305::aead::{Aead, KeyInit, Payload};
7  use chacha20poly1305::{ChaCha20Poly1305, Key, Nonce};
```



用计数器拼出 nonce



channel.rs

crates/lanshare-proto/src/

第 1-14 行 • 共 14 行

```
1  //! 加密信道: ChaCha20-Poly1305, nonce 由请求计数器构成。
2  //!
3  //! 每个方向一把密钥 (c2s / s2c), 每个请求的计数器唯一, 所以 (密钥, nonce) 永不重复—
4  //! 这是 ChaCha20-Poly1305 安全性的前提。
5
6  use chacha20poly1305::aead::{Aead, KeyInit, Payload};
7  use chacha20poly1305::{ChaCha20Poly1305, Key, Nonce};
8
9  /// 12 字节 nonce = 4 字节 0 + 8 字节大端计数器。
10 fn nonce(ctr: u64) -> Nonce {
11     let mut bytes = [0u8; 12];
12     bytes[4..].copy_from_slice(&ctr.to_be_bytes());
13     Nonce::from(bytes)
14 }
```



加密：seal



channel.rs

crates/lanshare-proto/src/

第 2-21 行 · 共 21 行

```

2  //!
3  //! 每个方向一把密钥 (c2s / s2c)，每个请求的计数器唯一，所以（密钥，nonce）永不重复—
4  //! 这是 ChaCha20-Poly1305 安全性的前提。
5
6  use chacha20poly1305::aead::{Aead, KeyInit, Payload};
7  use chacha20poly1305::{ChaCha20Poly1305, Key, Nonce};
8
9  /// 12 字节 nonce = 4 字节 0 + 8 字节大端计数器。
10 fn nonce(ctr: u64) -> Nonce {
11     let mut bytes = [0u8; 12];
12     bytes[4..].copy_from_slice(&ctr.to_be_bytes());
13     Nonce::from(bytes)
14 }
15
16 /// 加密并附上认证标签。返回 `明文长度 + 16` 字节。
17 pub fn seal(key: &[u8; 32], ctr: u64, aad: &[u8], plain: &[u8]) -> Vec<u8> {
18     ChaCha20Poly1305::new(Key::from_slice(key))
19         .encrypt(&nonce(ctr), Payload { msg: plain, aad })
20         .expect("ChaCha20-Poly1305 只有在明文超过 256 GiB 时才会加密失败")
21 }

```



四种 “一串字节”

写法	意思	打个比方
[u8; 32]	正好 32 个字节的数组，长度是类型的一部分	一个固定 32 格的盒子
&[u8; 32]	借用一个 32 字节数组，只读，不拿走	借来看看那个盒子
&[u8]	借用任意长度的一段字节，叫切片	借来看一段，长度运行时才知道
Vec<u8>	自己拥有的、能变长的字节数组	自己买的伸缩箱子





lib.rs: 声明模块，挑出公开的名字

lib.rs crates/lanshare-proto/src/

第 11-30 行 · 共 30 行

```

11 pub const PROTOCOL: &str = "lanshare/2";
12 /// ChaCha20-Poly1305 认证标签长度: 密文 = 明文 + 16 字节。
13 pub const TAG_LEN: usize = 16;
14
15 /// 协议层面的错误。刻意不区分“密钥错”“被篡改”“计数器错”: 对攻击者一律只说解不开。
16 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
17 pub enum ProtoError {
18     /// AEAD 认证失败: 密钥、计数器、aad 或密文任意一处不对。
19     Decrypt,
20 }
21
22 impl std::fmt::Display for ProtoError {
23     fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
24         match self {
25             ProtoError::Decrypt => write!(f, "解密失败"),
26         }
27     }
28 }
29
30 impl std::error::Error for ProtoError {}

```



解密：open，和附加数据

channel.rs crates/lanshare-proto/src/

第 28–47 行 • 共 47 行

```
28         .decrypt(&nonce(ctr), Payload { msg: sealed, aad })
29         .map_err(|_| ProtoError::Decrypt)
30     }
31
32     fn aad(prefix: &str, path: &str) -> Vec<u8> {
33         let mut out = Vec::with_capacity(prefix.len() + path.len());
34         out.extend_from_slice(prefix.as_bytes());
35         out.extend_from_slice(path.as_bytes());
36         out
37     }
38
39     /// 请求的附加认证数据：把密文绑定到具体接口，防止挪到别的接口重放。
40     pub fn req_aad(path: &str) -> Vec<u8> {
41         aad("lanshare/2 req ", path)
42     }
43
44     /// 响应的附加认证数据：请求和响应用不同前缀，密文不能互换。
45     pub fn resp_aad(path: &str) -> Vec<u8> {
46         aad("lanshare/2 resp ", path)
47     }
```



解密失败：异常 vs Result

Python (假如用 cryptography 库)

```
1 def open_(key, ctr, aad, sealed):
2     nonce = bytes(4) + ctr.to_bytes(8, "big")
3     return ChaCha20Poly1305(key).decrypt(nonce, sealed, aad)
4
5 try:
6     plain = open_(key, 5, aad, sealed)
7 except InvalidTag:
8     ... # 解不开
```

Rust v2 • channel.rs

```
1 pub fn open(key: &[u8; 32], ctr: u64, aad: &[u8], sealed: &[u8])
2     -> Result<Vec<u8>, ProtoError>
3
4 match open(&key, 5, &aad, &sealed) {
5     Ok(plain) => { /* 用明文 */ }
6     Err(ProtoError::Decrypt) => { /* 解不开 */ }
7 }
```



测试：任何一处不对都要失败



channel.rs

crates/lanshare-proto/tests/

第 11-30 行 · 共 30 行

```
11     let sealed = seal(&K, 42, b"aad", &plain);
12     assert_eq!(sealed.len(), len + 16, "密文 = 明文 + 16 字节认证标签");
13     assert_eq!(open(&K, 42, b"aad", &sealed).unwrap(), plain);
14 }
15 }
16
17 #[test]
18 fn any_mismatch_fails_to_open() {
19     let sealed = seal(&K, 5, &req_aad("/api/rpc"), b"secret");
20     assert_eq!(open(&[8; 32], 5, &req_aad("/api/rpc"), &sealed), Err(ProtoError::Decrypt), "错 key");
21     assert_eq!(open(&K, 6, &req_aad("/api/rpc"), &sealed), Err(ProtoError::Decrypt), "错计数器");
22     assert_eq!(open(&K, 5, &req_aad("/api/up/x/0"), &sealed), Err(ProtoError::Decrypt), "挪到别的接口");
23     assert_eq!(open(&K, 5, &resp_aad("/api/rpc"), &sealed), Err(ProtoError::Decrypt), "请求当响应");
24     for i in 0..sealed.len() {
25         let mut tampered = sealed.clone();
26         tampered[i] ^= 0x80;
27         assert_eq!(open(&K, 5, &req_aad("/api/rpc"), &tampered), Err(ProtoError::Decrypt), "篡改第 {i} 字节");
28     }
29     assert_eq!(open(&K, 5, &req_aad("/api/rpc"), &sealed[..10]), Err(ProtoError::Decrypt), "截断");
30 }
```



跑测试

```
Windows PowerShell

PS D:\code\lanshare> cargo test -p lanshare-proto
  Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
  Finished `test` profile [unoptimized + debuginfo] target(s) in 1.16s
  Running unittests src\lib.rs (target\debug\deps\lanshare_proto-dddfe3b7e4bfc739.exe)

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

  Running tests\channel.rs (target\debug\deps\channel-0f3ed991707f1028.exe)

running 2 tests
test any_mismatch_fails_to_open ... ok
test roundtrip_including_empty_and_large ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

  Doc-tests lanshare_proto

running 0 tests
```





踩坑：trait 没有引入

Windows PowerShell

```
PS D:\code\lanshare> cargo build -p lanshare-proto
Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
error[E0599]: no method named `encrypt` found for struct `ChaChaPoly1305<C, N>` in the current scope
--> crates\lanshare-proto\src\channel.rs:21:10
   |
20 | /     ChaCha20Poly1305::new(Key::from_slice(key))
21 | |         .encrypt(&nonce(ctr), Payload { msg: plain, aad })
   | |_____^
..... (省略 6 行: 库里 encrypt 定义在哪)
      = help: items from traits can only be used if the trait is in scope
..... (省略 2 行)
help: trait `Aead` which provides `encrypt` is implemented but not in scope; perhaps you want to import it
   |
6 + use chacha20poly1305::aead::Aead;
   |
..... (省略 32 行: 另一条建议, 以及 decrypt 的同样错误)
error: could not compile `lanshare-proto` (lib) due to 2 previous errors
```



选择题

seal 加密 1000 个字节，得到的密文有多长？

A 1000 字节

B 1012 字节

C 1016 字节

D 1032 字节



选择题

seal 加密 1000 个字节，得到的密文有多长？

A 1000 字节

ChaCha20 本身不改变长度，但 seal 还会附上 16 字节的认证标签。

B 1012 字节

12 是 nonce 的长度。nonce 不随密文发送，双方都能从计数器算出来。

C 1016 字节

密文和明文一样长，再加上 16 字节的认证标签，一共 1016 字节。

D 1032 字节

32 是密钥的长度，密钥永远不会出现在密文里。



选择题

请求和响应用的是同一个计数器，nonce 完全一样。这样安全吗？

A 不安全，nonce 重复了

B 安全，因为两个方向用的是不同的密钥

C 安全，因为附加数据的前缀不一样



选择题

请求和响应用的是同一个计数器，nonce 完全一样。这样安全吗？

A 不安全，nonce 重复了

nonce 只要求在同一把密钥下不重复。

C 安全，因为附加数据的前缀不一样

附加数据不同不能代替 nonce 不重复。真正起作用的是两把不同的密钥。

B 安全，因为两个方向用的是不同的密钥

请求用 c2s，响应用 s2c，两把密钥不同，所以“密钥加 nonce”这个组合没有重复。



选择题

open 失败时，为什么只返回一种错误 Decrypt，不说具体错在哪？

- A 因为 Rust 的 Result 只能装一种错误
- B AEAD 本来就分不出错在哪，对外也不该多说
- C 为了让代码短一点



选择题

open 失败时，为什么只返回一种错误 Decrypt，不说具体错在哪？

A 因为 Rust 的 Result 只能装一种错误

Result 的错误类型可以是任何枚举，下一集 ProtoError 就会多一种 Pake。

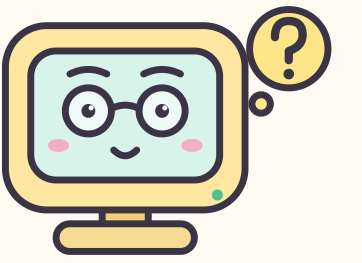
C 为了让代码短一点

少几行代码不是理由，这是一个安全上的选择。

B AEAD 本来就分不出错在哪，对外也不该多说

AEAD 本身只知道标签对不上，分不出是密钥错、计数器错还是被篡改。我们也不在别处补充细节，对外一律只说解不开，不给攻击者任何线索。

想一想



seal 里用 expect，失败就让程序崩溃；open 里却用 map_err，把错误交给调用方。为什么两边处理方式不一样？



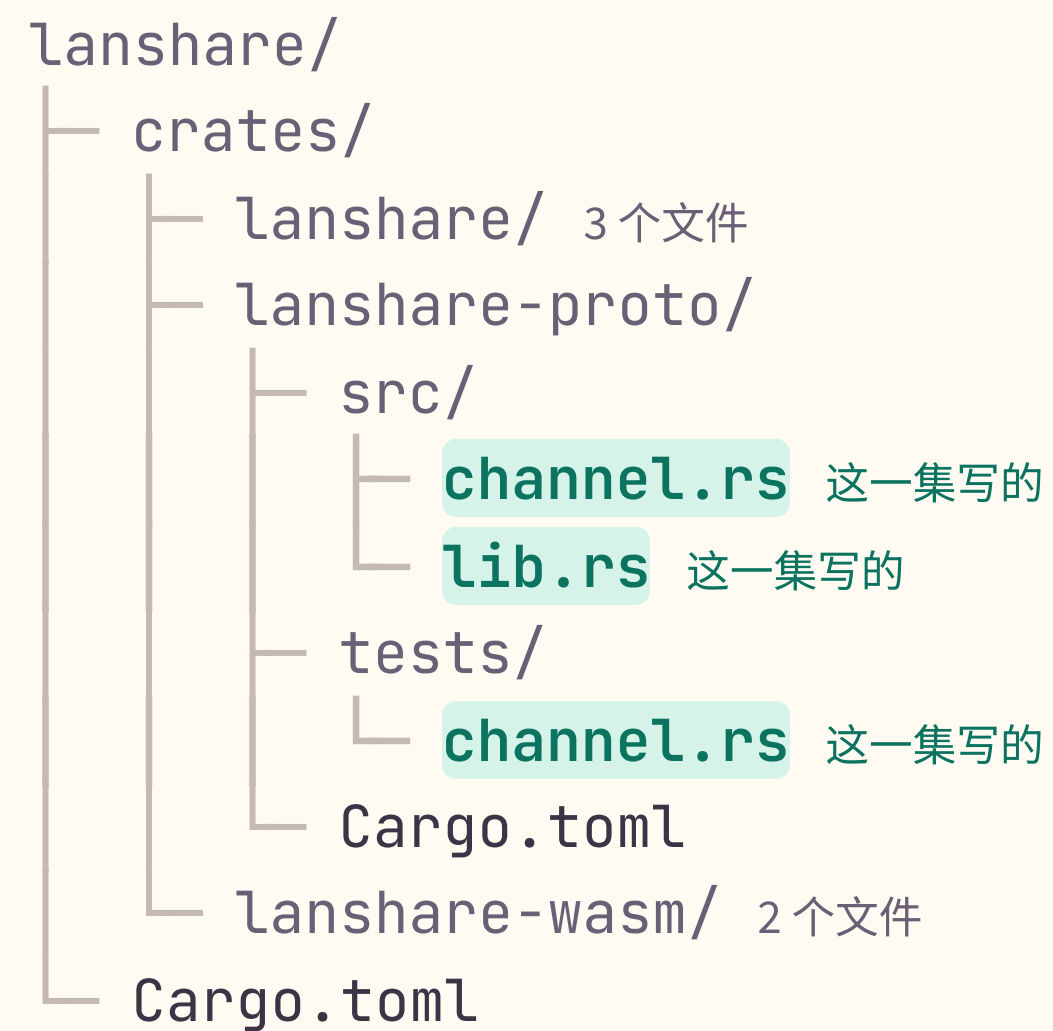
想一想

seal 里用 expect，失败就让程序崩溃；open 里却用 map_err，把错误交给调用方。为什么两边处理方式不一样？

加密失败只有一种可能：明文超过 256 GiB。我们一块最多 1 MiB，这是写代码的人能保证永远不会发生的事，所以用 expect 写明理由。解密失败却是由外面的输入决定的：密文被篡改、密钥不对、有人乱发请求，随时都会发生。它是正常情况，不能让服务端崩溃，必须作为 Result 交给调用方处理。



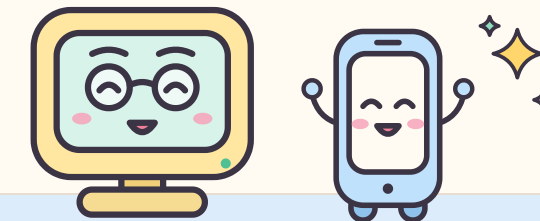
这一集写完了





这一集写了什么

- ✓ AEAD 一次完成加密和防篡改：密文 = 明文 + 16 字节标签
- ✓ nonce = 4 个零字节 + 8 字节大端计数器；两个方向两把密钥，组合永不重复
- ✓ 附加数据绑定接口路径和方向，密文挪不走
- ✓ & 是借用，Vec 是自己拥有；会失败的函数返回 Result
- ✓ 集成测试：错密钥、错计数器、换接口、篡改每一个字节、截断，全都必须失败



下一集

全课完结，感谢收看